

Exemple : je veux stocker les notes des 300 étudiants

```
int main()
{
    int etudiant1 = 5;
    int etudiant2 = 10;
    ...
    int etudiant300 = 8;

    double moyenne = moyenne (etudiant1, ... etudiant300);
}

double moyenne (int etudiant1, int etudiant2, ... int etudiant300)
{
    double moyenne = etudiant1 + etudiant2 + ... + etudiant300;
    moyenne = moyenne / 300;
}
```

Avec des variables

- En créer manuellement 300...
- Toutes les passer aux fonctions
- Toutes les écrire dans les calculs

```
int main()
{
    int tab[300] = {5,10,...,8};
    double moyenne = moyenne (tab);
}

double moyenne (int tab[])
{
    double moyenne = 0;
    for (int i = 0; i < 300 ; i = i+1)
    {
        moyenne = moyenne + tab[i];
    }
    moyenne = moyenne / 300;
}
```

Avec un tableau

- Une seule variable à créer
- Une seule variable à passer aux fonctions
- Une seule boucle pour les traiter toutes

Déclarer un tableau statique

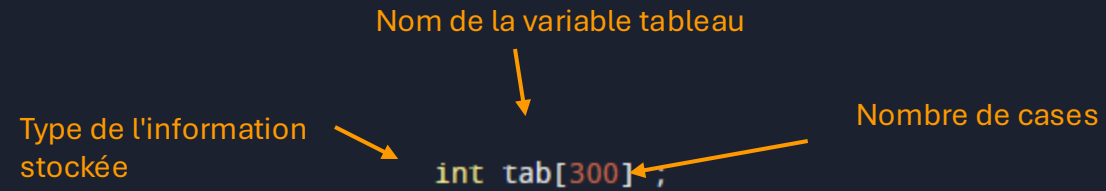
Une suite de cases pouvant contenir chacune une information.



Une seule dimension

Type de l'information stockée Nom de la variable tableau Nombre de cases

```
int tab[300];
```



Initialiser (au moment de la déclaration)

Initialisation totale

Je donne toutes les valeurs, séparées par des virgules.

Initialisation partielle

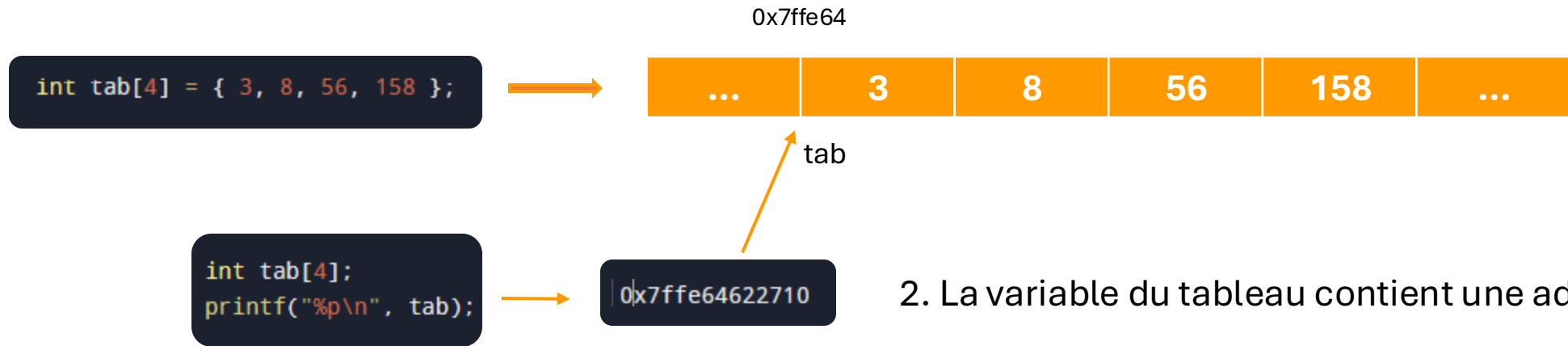
Je ne donne que les premières, toute la suite est mise à 0.

```
int tab[5] = { 3 , 5 , 8 , 12 , 8 };  
double tab[3] = { 3.5 , 8.2 , 7.3};  
bool tab[4] = { 0 , 1 , 1 , 1 };
```

```
int tab[5] = { 3 , 5 , 8 };      3,5,8,0,0  
double tab[3] = { 3.5 };        3.5,0.0,0.0  
bool tab[4] = { 0 , 1 };        0,1,0,0
```

Comment passe-t-on de 300 variables à une seule ?

1. Un tableau réserve des espaces mémoires contigus (les uns à côté des autres).



2. La variable du tableau contient une adresse !

-> Un tableau se comporte comme un pointeur vers le premier espace mémoire
(Ça pourrait ressembler à un pointeur constant car on ne peut plus changer l'adresse qui est stockée dedans)

3. Pour accéder aux autres cases : décaler l'adresse d'un espace mémoire (pour ça qu'ils sont côte à côte !)

Comment accéder aux données ? (notation pointeur – ne pas utiliser)



On peut manipuler un tableau comme un pointeur

→ `printf("%d\n", *tab );` → 3
ou `printf("%d\n", *(tab+0) );`

Pour la case 2 : décaler l'adresse de 1 espace mémoire
(autant de bits qu'il faut pour le type de variable déclaré)

→ `printf("%d\n", *(tab+1) );` → 8

N'ajoute pas 1 à l'adresse mais ce qu'il faut pour décaler de 1 espace mémoire



La "numérotation" commence à 0 pour la 1ère case, puis 1 pour la seconde case, etc.

Pour la case x : $*(tab + x - 1)$

Pour modifier la valeur :

`*(tab + 3 ) = 14;`

Mets la valeur 14 dans la 4ième case

Comment accéder aux données ? (notation en tableau - version conseillée)



Première case :

```
printf("%d\n", tab[0] );
```

3

Deuxième case :

```
printf("%d\n", tab[1] );
```

8

Case x :

```
printf("%d\n", tab[x-1] );
```



La "numérotation" commence à 0 pour la 1ère case, puis 1 pour la seconde case, etc.

Le nombre dans les crochets dit de combien il faut décaler le pointeur.

Donc pour la première case on ne décale pas. Pour la seconde on décale de 1.

Modifier la valeur :

```
tab[3] = 14 ;
```

Mets la valeur 14 dans
la 4ème case

Initialiser (2)

Initialisation à la déclaration (rappel)

```
int tab[5] = { 3 , 5 , 8 , 12 , 8 };  
double tab[3] = { 3.5 , 8.2 , 7.3 };  
bool tab[4] = { 0 , 1 , 1 , 1 };
```

Totale

```
int tab[5] = { 3 , 5 , 8 };  
double tab[3] = { 3.5 };  
bool tab[4] = { 0 , 1 };
```

Partielle

Initialisation après la déclaration

```
int tab[5];  
tab[0] = 0;  
tab[1] = 2;  
tab[2] = 4;  
tab[3] = 6;  
tab[4] = 8;
```

```
int tab[5];  
int i ;  
for ( i=0; i < 5; i = i+1 )  
{  
    tab[i] = i *2;  
}
```

```
int tab[300];  
int i ;  
for ( i=0; i < 300; i=i+1 )  
{  
    scanf("%d", &tab[i] );  
}
```

Taille d'un tableau

```
int tab[300];
```

Taille "en dur" dans le code

La taille d'un tableau est statique, écrite directement dans le code.

Impossible de demander à l'utilisateur la taille.

```
int taille;  
scanf("%d", &taille);  
int tab[taille];
```

Par contre il est important de stocker la taille du tableau tout de même.

```
int etudiants[215];  
int tailleEtudiants = 215;  
  
...  
  
for (int i = 0; i < tailleEtudiants; i = i + 1)  
{  
    printf("%d\n", etudiants[i] );  
}
```


Taille d'un tableau réelle vs max

```
int tab[300];
```

Taille "en dur" dans le code

```
int taille;  
scanf("%d", &taille);  
int tab[taille];
```

Impossible de demander à l'utilisateur la taille.

Réserver beaucoup pour couvrir tous les cas

Ex : pour fonctionner avec toutes les promos je prends 2000 cases.

Conserver la taille max et la taille réelle

Ex : le tableau fait 2000 cases mais dans ma promo je n'ai que 215 élèves, donc j'utilise que les 215 premières.

```
int etudiants[2000];  
int tailleEtudiantsMax = 2000;  
int tailleEtudiantsReelle = 215;  
scanf("%d", &tailleEtudiantsReelle)  
  
for (int i = 0; i < tailleEtudiantsReelle; i = i + 1)  
{  
    printf("%d\n", etudiants[i] );  
}
```

Application dans les fonctions : passage en paramètre



Passer le tableau et sa taille !

```
int main()
{
    int tab[4] = { 2 , 5 ,8, 9 };
    affiche(tab, 4);
}
```



```
void affiche ( int tab[], int taille)
{
    int i;
    for (i = 0; i < taille; i = i +1)
    {
        printf("%d\n", tab[i]);
    }
}
```

Possible aussi car ça peut se manipuler
comme un pointeur

```
void affiche ( int *tab, int taille)
{
    int i;
    for (i = 0; i < taille; i = i +1)
    {
        printf("%d\n", tab[i]);
    }
}
```

Application dans les fonctions : portée des variables

Puisque l'on manipule des adresses (comme un pointeur) si on modifie le tableau dans la fonction il sera modifié partout (puisque l'on accède aux mêmes cases mémoires).

```
void saisieTableau( int tableau[], int taille)
{
    int i;
    for (i=0; i< taille; i=i+1)
    {
        scanf("%d", &tableau[i]);
    }
}
```

```
int main()
{
    int tab[4];
    saisieTableau(tab, 4);
    printf("%d\n", tab[1]);
}
```



Application dans les fonctions : retourner un tableau (à éviter, passer le plutôt comme paramètre)

Un tableau se "retourne" comme un pointeur.



Si le tableau est créé dans la fonction : il est détruit à sa sortie donc le pointeur retourné pointe sur un endroit invalide (car désalloué).

```
int main()
{
    int* tab = creerTableauChiffres();
    printf("%d\n", tab[4]);
}
```

L'espace mémoire est désalloué en
sortant de la fonction



```
int * creerTableau()
{
    int tab[5]= {12, 15, 8, 1, 3};
    return tab;
}
```

Tableau multidimensionnel

Déclaration

2D

```
int tab[3][4]
```

3D

```
int[3][5][2]
```

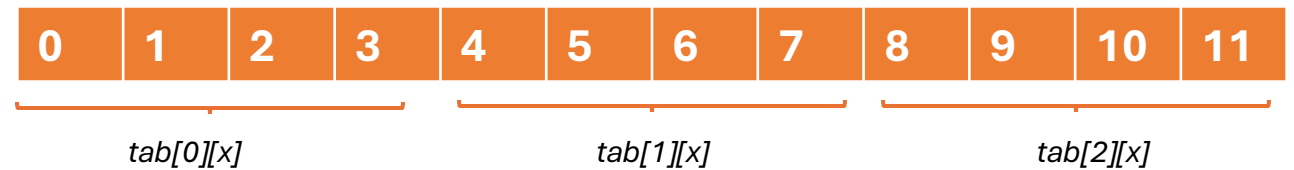
Initialisation

```
int tab[3][4] = { {0, 1, 2, 3} , {4, 5, 6, 7} , {8, 9, 10, 11} };
```

```
int tab[3][5][2] = { { {0, 1}, {2, 3}, {4, 5}, {6, 7}, {8, 9} },  
                     { {10, 11}, {12, 13}, {14, 15}, {16, 17}, {18, 19} },  
                     { {20, 21}, {22, 23}, {24, 25}, {26, 27}, {28, 30} } },  
                     };
```

<i>tab[0][x]</i>	0	1	2	3
<i>tab[1][x]</i>	4	5	6	7
<i>tab[2][x]</i>	8	9	10	11

Représentation matricielle



En mémoire

Tableau multidimensionnel

Parcours du tableau avec des boucles imbriquées

```
int tab[3][4]
```

```
int i, j;
for (i =0; i<3; i=i+1)
{
    for (j=0; j<4; j=j+1)
    {
        printf("%d\n", tab[i][j]);
    }
}
```

```
int tab[3][4][2]
```

```
int i, j, k;
for (i =0; i<3; i=i+1)
{
    for (j=0; j<4; j=j+1)
    {
        for(k=0; k<2; k=k+1)
        {
            printf("%d\n", tab[i][j][k]);
        }
    }
}
```

Passage dans une fonction

```
void saisieTableau (int tableau[][[]], int tailleX, int tailleY)
void saisieTableau (int ** tableau, int tailleX, int tailleY)
```